eosc-basic-compliance

Download, installation, configuration and run guide

Verified against a clean clone on 26 September 2026 — rebuilt 20 October 2026 for run web-33 and EOSC Node Germany (re-issued checklist v3.2)

Repository: github.com/marioreale/eosc-basic-compliance

eosc-basic-compliance checks EOSC Node Landing Pages against the **Node Landing Page Verification Checklist v3.2** (28 September 2026) and writes a report: a matrix of nodes against checklist points, with the evidence behind every verdict.

This guide covers installing it, configuring it for your own nodes, running it, reading what comes out, and running the test suite. This edition was checked against a clean clone of commit ada1b4a on 25 September 2026, and the test count again at commit 7ed46dd on 26 September 2026. Every command that needs no network — installation, the test suite, points, assess and the rebuild of the published report — was re-run that day, and the test counts, disk sizes and verdict tallies were re-measured rather than carried over. No node website was contacted to prepare it. Request counts and HTTP behaviour come from the runs of 24 and 26 September and from a trial run on 25 September, and each figure says which. Where a command's behaviour is surprising, that is noted rather than smoothed over.

This edition was revised on 20 October 2026, when EOSC Node Germany was added and run web-33 was published, after a per-node time limit (--node-timeout, section 4) was added the same day; the test count is 462. The edition of 19 October 2026 published run web-31 after PaNOSC's landing page changed; its tallies and the rebuild of the published report were re-measured offline that day. The previous edition, of 9 October 2026, added EBRAINS-RI and Life Sciences Connect with run web-28. The nodes were contacted only by the GitHub workflow; a hand check of Life Sciences Connect's pages, made from a browser, is reported where it is used.

> **Checklist v3.2 since 29 September 2026.** The default reference checklist is now v3.2 of 28 September 2026 (checklist/v3.2.yaml). Only item 5 changed: the purpose description, the Acceptable Use Policy and the User Access Policy of every Node Exchange resource must now be available **both** on the NLP, directly or via intermediate web pages linked by the NLP, **and** in the resource's EOSC Catalogue metadata, where v3.1 asked for either. The document was re-issued on 29 September 2026 with this wording; the first copy, which required them "directly on the NLP", is discarded and its hash replaced. So a pointer on the landing page, or on a policies, legal, services or resources page it links to, settles the NLP half, and the crawler follows such links (purposes 5p and 5s). A landing page with no pointer anywhere the tool read **fails** 5b or 5c, unless it links to such a page the tool did not read, which is review. Two notes in the new item 5 also keep review rather than FAIL: one document may serve as both AUP and UAP, and datasets, archives and software may provide them through a product licence. The rule is selected by the checklist file (item5_on_nlp), so runs made under v3.0 and v3.1 still rebuild exactly with -c. The header of checklist/v3.2.yaml lists the differences point by point, and section 5 of the analysis workflow has the decision procedure.
>
> **The published run changed again on 20 October 2026.** results/ now holds run web-33, collected by the GitHub workflow at --depth 2 on 20 October 2026 between 20:13 and 20:21 UTC from all sixteen nodes (run 39489960802, commit 1353fda), under the re-issued v3.2: 81 PASS, 7 FAIL and 72 MANUAL_REVIEW at depth 2 (79 / 7 / 74 at depth 1), in 130 page requests. It adds **EOSC Node Germany** (eosc-de, https://eosc-node.de, approved name "EOSC Node | Germany"): 3 PASS, 4 review and 3 FAIL (5b, 5c, 6) on a "Coming soon" page with no policy or contact link. No cell of the

other fifteen nodes changed against web-31. web-32 (run 39387128370), the first run with the per-node time limit, was not published: EBRAINS-RI hit the limit and read ERROR. It has **not** been reviewed by hand.

Previously, on 19 October 2026, results/ held run web-31, collected by the GitHub workflow at --depth 2 on 19 October 2026 between 20:44 and 20:51 UTC from all fifteen nodes (run 39321743220, commit e9c5f9c), under the re-issued v3.2: 78 PASS, 4 FAIL and 68 MANUAL_REVIEW at depth 2 (76 / 4 / 70 at depth 1), in 129 page requests. PaNOSC's landing page changed that day to https://panosc.eu/panosc-eosc-node; the old address redirected to the same page, so its verdicts did not change. At depth 2 no cell changed against web-28; at depth 1 Poland point 6 moved from PASS to review (its landing page now shows contact@eosc.pl, not support@eosc.pl). It has **not** been reviewed by hand.

Previously, on 9 October 2026, results/ held run web-28, collected by the GitHub workflow at --depth 2 on 9 October 2026 between 14:24 and 14:32 UTC from all **fifteen** nodes (run 37943784224, commit df4be09), under the re-issued v3.2: 78 PASS, 4 FAIL and 68 MANUAL_REVIEW at depth 2 (77 / 4 / 69 at depth 1), in 129 page requests. Two nodes were added that day: EBRAINS-RI (https://ebrains.eu/) and Life Sciences Connect (https://eosc-lifesciences-connect.eu/home), with their approved names added to both name lists. All four FAILs are theirs: EBRAINS-RI point 4 (no link to eosc.eu), and Life Sciences Connect 5b, 5c and 6. Life Sciences Connect's menu items are <a> elements without href, so the tool sees two links only; its point 6 FAIL is a false negative (a hand check found help@eosc-lifesciences-connect.eu on its Contact page), and its 5b/5c FAILs need a reviewer's confirmation. On the other thirteen nodes, one cell changed: EUDAT point 6, review to PASS. It has **not** been reviewed by hand.

Before that, on 7 October 2026, results/ held run web-27, collected by the GitHub workflow at --depth 2 on 7 October 2026 between 21:59 and 22:05 UTC from all thirteen nodes (run 37692836637, commit 5e0e6ce), under the re-issued v3.2: 69 PASS, 0 FAIL and 61 MANUAL_REVIEW at both depths, in 109 page requests. Two cells changed, both Poland's: its page rendered in full again, so point 4 (its own eosc.eu entry) passes again, and 5c passes through a new "User Access Policy" link to its privacy policy page, which a reviewer should confirm. GÉANT got the Cloudflare challenge again. Before it came web-26 (run 37492974153, 6 October, 67 / 0 / 63), in which Poland's page rendered only its header and footer, and web-25 (6 October, morning, 68 / 0 / 62), the first run with no FAIL. web-14 was the first run that reads the policies and services pages linked from the landing pages. CERN's 5b and 5c pass through its Policies page; Data Terra's fail after its services pages were read (a French-named policy would be missed, so check it by hand); PaNOSC's are review. It has **not** been reviewed by hand. Before it, results/ briefly held web-13-rescored, the evidence of web-13 scored offline against the re-issued v3.2 (50 PASS, 8 FAIL, 72 MANUAL_REVIEW).

Before that, results/ held run web-13, collected by the GitHub workflow at --depth 2 on 28 September 2026 between 22:53 and 22:57 UTC from all thirteen nodes (run 36494907672), under the first copy of v3.2: 49 PASS, 14 FAIL, 67 MANUAL_REVIEW at both depths, since the 19 second-hop pages changed no verdict. It repeats cell for cell web-12 (run 36493197701), the first collection under v3.2, at depth 1, and web-11-v3.2, the evidence of run web-11 (run 36431202625) scored again against v3.2 without contacting any node, which was published briefly before it. Under v3.1 the same evidence gave 49 PASS, 8 FAIL and 73 MANUAL_REVIEW; the six new FAILs are 5b and 5c for CERN, Data Terra and PaNOSC. web-11 was the first run with CERN's new landing page, https://eosc.cern/, and followed web-10 of the same day, the first with EOSC Node Poland, which replaced a node listed by mistake. It has **not** been reviewed by hand; the review of the run of 24 September stays in results/REVIEW-2026-09-24.md as history. Passages below that describe "the published run of 24 September" are about that earlier, reviewed run, whose files are in the git history at commit 800d632.

What this tool will not do: it does not produce a compliance statement. Of the ten checklist points, three can be settled by inspection, four only partly, and three require a human reading the page. The tool returns `MANUAL_REVIEW` rather than guessing, and roughly 60% of cells land there by design. A checker that returned PASS/FAIL on every line would look more useful and be worth less.

👉 **Analysis workflow** — the companion to this guide. This one tells you how to run the tool; that one tells you what it does once running: the collection sequence, the evidence model, and for each of the ten checklist points the exact branch conditions and the verdict each one yields. Read it when you want to predict a verdict, argue with one, or change a check.

---

1. Requirements

Python	3.12 or newer (the project is developed on 3.14)
Package manager	uv
Browser	Chromium, via Playwright — **only needed to collect evidence**
Disk	~500 MB for the virtualenv, ~660 MB more for Chromium (see the note below)
Network	Outbound HTTPS to the node websites, for collection only

Installing uv, if you do not have it:

```
curl -LsSf https://astral.sh/uv/install.sh | sh    # macOS / Linux
```

You do not need to create a virtualenv or run `pip install`. uv manages the environment from `uv.lock`, so everyone gets identical dependency versions.

On the disk figures. Both were measured with `du -sh`, not estimated. The 500 MB is `.venv` in the project directory (484 MB in a fresh clone on 25 September 2026, rising to about 510 MB once caches accumulate). The 660 MB is one Playwright install — Chromium (393 MB), the headless shell (261 MB) and ffmpeg (5 MB) — and it does **not** live in the project: Playwright puts browsers in a shared cache at `~/.cache/ms-playwright`, so a second project on the same machine reuses it and costs nothing. That cache can nonetheless grow well past 660 MB, because each Playwright version keeps its own browser build; the machine this was measured on held two versions and so 1.3 GB in total. Delete the whole directory and re-run `playwright install` to reclaim it.

---

2. Installation

```
git clone https://github.com/marioreale/eosc-basic-compliance.git
cd eosc-basic-compliance
uv sync --locked
```

This creates `.venv/` and installs exactly the versions pinned in the committed `uv.lock`: it resolves 27 packages and installs 26, the 27th being the project itself. Expect it to take under a minute; from a warm uv cache it took two seconds. A plain `uv sync` also works, but it will silently re-resolve if the lockfile and `pyproject.toml`

ever disagree; --locked fails instead, which is what you want when the point is to reproduce someone else's result. CI uses --locked for the same reason (section 8).

At this point you can already run the test suite, read the checklist, and regenerate the committed report from the evidence in the repository. To **fetch pages**, you also need a browser:

```
uv run playwright install chromium --with-deps
```

This downloads roughly 660 MB (see the note in section 1). On Linux, --with-deps also installs system libraries and will ask for sudo. If you only want to re-render reports from evidence already in the repository, skip this step.

What needs a browser, and what does not

This distinction saves a large download in CI and on review machines:

Command	Browser	Network	What it does
points	no	no	Prints the checklist and what is machine-decidable
assess	no	no	Applies the checklist to evidence already on disk
show	no	no	Prints one node's results in the terminal
collect	**yes**	**yes**	Fetches pages and saves evidence
run	**yes**	**yes**	collect, then assess
pytest	no	no	The whole test suite is offline

Verified: with PLAYWRIGHT_BROWSERS_PATH pointed at an empty directory, all 462 tests still pass, while collect fails with Playwright's Executable doesn't exist … run playwright install.

Verifying the installation

```
uv run pytest -q              # expect: 462 passed
uv run ruff check src tests   # expect: All checks passed!
uv run basic-check points     # prints the ten checklist points
```

If pytest passes, the checklist transcription, the check logic and the report renderer are all working. No network access is involved.

---

3. Configuration

Two files control what is checked, and a third optional one supplies an input the repository cannot ship. The two are plain YAML and both are meant to be edited; the third is a plain text list you write yourself.

nodes.yaml — which pages to check

```
nodes:
  - id: bbmri-eric
```

```
name: BBMRI-ERIC
url: https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/
eosc_page: https://eosc.eu/building-the-eosc-federation/eosc-node-bbmri-eric/
```

Field	Meaning
id	Short slug. Used for evidence filenames and for --only / show.
name	Display name in the report.
url	**The URL registered in the EOSC EU Node Contributors Dashboard** ("Website address", section 1.2 field 6). That registered URL is what the checklist defines as the Node Landing Page. If the URL here differs from the registered value, the registered value wins — correct this file.
eosc_page	The node's dedicated entry on eosc.eu, which checklist point 4 requires the landing page to link to.

eosc_page is what lets a point 4 failure name the exact URL that is missing, rather than reporting a bare absence. The slugs in the committed file were read from the live index at eosc.eu/building-the-eosc-federation/. If you add a node, look its slug up there rather than guessing it from the name.

BBMRI-ERIC's URL has changed several times. On 25 September 2026 it was changed to https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/; on 26 September it was set back to the www. address, because the dev3. host's robots.txt excludes every path. On 1 October it was changed to the dev3. address again, and set back the same day for the same reason (run web-18 found the page disallowed). On 5 October 2026 the dev3. address shown above was made the default (commit 42aacbe), and nodes.yaml carries a comment recording all of this. Run web-24 (5 October) was collected from it while the page was still disallowed, so all ten BBMRI-ERIC points were ERROR (section 6). By run web-25 (6 October), the published run, the host's robots.txt no longer excluded the tool, and the page was read. Its last evidence from the www. page is that of web-23, in the git history at 73fbb93.

EOSC Node Italy's URL changed twice. On 26 September 2026 it changed from https://eosc.it/ to https://eoscnode-it.d4science.org/, because eosc.it had no address record on 24 September; Italy was assessed from that address in runs web-4 to web-16. On 1 October it changed back to https://eosc.it/, which now resolves, and nodes.yaml carries a comment saying so. Runs web-18, web-21 to web-33 (the published run) were collected from https://eosc.it/.

Data Terra's URL changed on 1 October 2026, from https://www.data-terra.org/eosc/ to https://www.earth-data.eu/, and nodes.yaml carries a comment saying so. Runs up to web-18 read the old address; runs web-21 to web-33 (the published run) read the new one.

PaNOSC's URL changed on 19 October 2026, from https://eosc.panosc.eu/ to https://panosc.eu/panosc-eosc-node, with basic-check --update-nlp (commit e9c5f9c); nodes.yaml and the shipped default copy carry the same comment. Both addresses redirect to https://www.panosc.eu/panosc-eosc-node/, which is the final_url of every run since the redirect appeared, so the change records the node's chosen address rather than a different page. Runs up to web-28 were requested at the old address; web-31 and web-33 (the published run) at the new one.

EOSC Node Germany was added on 20 October 2026 (commit 1353fda): id eosc-de, URL https://eosc-node.de, EOSC page https://eosc.eu/building-the-eosc-federation/eosc-node-germany/, and the approved name "EOSC Node | Germany" in both checklist/approved-names.txt and the scoped list. It was first collected in web-33.

Changing a node's URL

Change url in nodes.yaml and run uv run pytest -q. Nothing else in the repository stores the URL. You can edit the file by hand, or let the tool do it:

```
uv run basic-check --update-nlp bbmri-eric https://new.example.org/eosc-node/
```

--update-nlp takes the node id (the id exactly, not the name) and the new URL. It changes only that node's url: line, adds a comment above it with the date and the old URL, and keeps every other line and comment as it was. It refuses a URL that is not absolute https, one that is already another node's landing page, and an unknown id, and it prints the old and new URL. It contacts no site, commits nothing and leaves results/ alone; check the change with git diff nodes.yaml and commit it yourself.

To undo it, and any other edit to nodes.yaml, put the file back as it was downloaded from GitHub:

```
uv run basic-check --restore-default-config
```

The replaced file is kept beside it as nodes.yaml.<date-time>.bak (ignored by git), and the output lists every node whose URL was restored, added back or removed. If the file is already the default, nothing changes. The default is a copy shipped inside the package (src/basic_check/defaults/nodes.yaml), so this works without git, from a zip download too. Only nodes.yaml is restored: the approved-name lists in checklist/ are not touched. Like --update-nlp, it contacts no site, commits nothing and leaves results/ alone; for a node whose URL it restored, run --update-results-for-node <id> re-checks that node. To re-check only that node and update only its row in the results, keeping the others, use run --update-results-for-node (section 4). To look first without changing anything published, collect it into a scratch directory first, so you can see what the new page gives before anything published changes:

```
uv run basic-check collect --only bbmri-eric --results /tmp/trial
uv run basic-check assess  --only bbmri-eric --results /tmp/trial
```

assess takes each node's URL from nodes.yaml, not from the evidence. So after a URL change, a bare uv run basic-check assess pairs the new URL with evidence collected from the old page. It no longer does so silently. assess compares each node's configured URL with the requested_url stored in its evidence. When they differ, it prints a warning naming the node and both addresses, adds an **Evidence from a different URL** banner to results.md and index.html, and lists the pairs under url_mismatch in results.json. The exit code does not change, because the table is complete; it is just not about the configured page. The address finally reached, after redirects, is kept as final_url in the evidence file.

To rebuild the published run, assess it under its own label. It was collected from the committed nodes.yaml and scored under the default checklist, so no other option is needed and there is no URL warning:

```
uv run basic-check assess --run web-33
```

The evidence records the depth it was collected at, so assess renders both depths again without any option.

Run web-11 is in the git history: restore its results/evidence from commit d3de829 into a scratch directory and assess it there with --results, as --run web-11-v3.2 for the v3.2 scoring (49 PASS, 14 FAIL, 67 MANUAL_REVIEW under the discarded first copy of v3.2; the current rules give other figures) or with -c checklist/v3.1.yaml --run web-11 for the scoring published on 28 September (49 PASS, 8 FAIL, 73 MANUAL_REVIEW).

Run web-10, collected before CERN's URL changed, needs its own node list (git show 8126aee:nodes.yaml) passed with --nodes.

Verified on 5 October 2026: this reproduces the committed results/ exactly, apart from the generation time, with 60 PASS, 1 FAIL, 59 MANUAL_REVIEW and 10 ERROR for web-24; verified again on 6 October 2026 for web-25, with 68 PASS, 0 FAIL and 62 MANUAL_REVIEW, and for web-26, with 67 PASS, 0 FAIL and 63 MANUAL_REVIEW; verified on 7 October 2026 for web-27, with 69 PASS, 0 FAIL and 61 MANUAL_REVIEW; on 9 October 2026 for web-28, with 78 PASS, 4 FAIL and 68 MANUAL_REVIEW at depth 2; on 19 October 2026 for web-31, with the same counts at depth 2; and on 20 October 2026 for web-33, with 81 PASS, 7 FAIL and 72 MANUAL_REVIEW. The reviewed run of 24 September that it replaced is in the git history at commit 800d632; to rebuild that one, restore its results/ into a scratch directory and assess it with --skip Italy and the node list from commit 53081f6. Runs published before 28 September 2026 were also scored with the earlier name list, so pass it too: git show e7da5a8:checklist/approved-names.txt > /tmp/names-before-2026-09-28.txt, then add --approved-names /tmp/names-before-2026-09-28.txt. The full procedure, including how to publish the new node's results, is example 2 in section 11.

A new URL can behave differently from the old one in ways that have nothing to do with the checklist. On the 25 September trial, dev3.bbmri-eric.eu served a robots.txt of User-agent: * / Disallow: /. The tool honours it, so no page was requested and all ten points for BBMRI-ERIC are ERROR (section 6). That is typical of a staging host and says nothing about compliance. Runs web-18 (1 October) and web-24 (5 October) found the same. By web-25 (6 October) the host allowed the tool and the page was read. If it happens, ask the node for the public address, or for the development host to allow the checker.

checklist/v3.2.yaml — the rules

One entry per checklist point. The requirement text is quoted verbatim from the source document so a reader can audit the rule without a diff, and decidable records whether a script can settle the point at all:

- true — decidable by inspection (points 1, 4, 7)
- partial — partly decidable; the tool gathers evidence and defers (3, 5b, 5c, 6)
- false — needs a human reading the page (1R, 2, 5a)

The file also pins its own provenance: source_file names the committed PDF the transcription was made from and source_sha256 is that file's hash, asserted by tests/test_checklist.py. v3.2 also carries item5_on_nlp: true, the one value the rules take from the file: it selects the item 5 rule of v3.2 (policies linked from the NLP or a page it links to, and in the Catalogue). v3.0.yaml and v3.1.yaml do not carry it, so they apply the older rule. Be clear about what this proves — it pins the exact bytes the transcription came from, which catches silent drift when a checklist is revised. It is not a signature, and the committed PDF is a rendering of the circulated .docx, not that .docx itself.

The checklist is not downloaded from anywhere. No file in the repository holds a URL for it. The document is committed in checklist/, and one line in src/basic_check/cli.py selects the revision used by default:

```
DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.2.yaml"
```

--checklist / -c overrides it for a single run. The help texts, the report file name (checklist-v3.2.html) and the tests all follow that line.

If the checklist is revised to v3.3, add a new checklist/v3.3.yaml with its own source file and hash rather than editing v3.2 in place, then change that line. Past runs should remain reproducible against the rules that produced them. The full procedure is example 3 in section 11.

The approved-names file — `--approved-names`

Checklist point 3 has two halves: an EOSC logo, and **the official Tripartite-approved node name**. The official list of those names is committed to the repository at checklist/approved-names.txt, and is used automatically. You do not have to supply anything for the name half to be assessed.

```
EOSC Node | BBMRI-ERIC
EOSC Node | CERN
EOSC Node | Czechia
EOSC Node | European DTO
EOSC Node | Data Terra
EOSC Node | Finland
EOSC Node | PaNOSC
EOSC Node | EUDAT
EOSC Node | EGI
EOSC Node | GÉANT
EOSC Node | Poland
EOSC Node | Italy
EOSC Node | Slovakia
```

Which list a run used is recorded in the report, in one line at the top of both the HTML and the Markdown, and in results.json under approved_names. A reader should never have to assume that the run they are looking at checked the names they think it did.

Which list is used

You run	List used
basic-check assess	the committed checklist/approved-names.txt
basic-check assess --approved-names mine.txt	mine.txt, instead of the default
basic-check assess --no-approved-names	none; point 3's name half is not assessed

A --approved-names path that does not exist is a clean error, **not** a silent fall back to the default. You named a specific list; checking a different one is not a smaller failure than checking none.

Both flags work identically on assess and on run.

Supplying your own list

One entry per line. Blank lines and # comments are ignored; write \# for a literal hash in a name.

```
# My own list, 21 September 2026.
bbmri-eric: EOSC Node - BBMRI-ERIC   # applies to that node only
EGI Node                             # applies to every node
```

Prefix each name with the node id from nodes.yaml. That ties the name to the node it belongs to, and it is the difference between the tool reporting "this page shows this node's approved name" and the much weaker "some approved name appears somewhere on this page". The committed official list is *not* scoped — it arrived as bare names, with nothing tying a name to a node — so runs using the default make the weaker claim, and say so. If you need the stronger one, copy the file, add the node-id: prefixes, and pass it with --approved-names.

```
uv run basic-check assess                          # official list
uv run basic-check assess --approved-names my-names.txt   # your list
uv run basic-check assess --no-approved-names          # no list at all
```

A file called approved-names*.txt **in the repository root** is gitignored, so a list you drop next to the project to override the default does not reach the repository by accident. The committed default, being under checklist/, is unaffected.

How a name is matched

Behaviour	Consequence
Matching is case-insensitive, and each word is **literal**	Regular-expression characters in a name are matched as themselves.
A name must not be part of a longer word	EGI matches "the EGI Foundation" but not "strat**egi**c". A hyphen counts as part of a word, so BBMRI does not match BBMRI-ERIC.
The separator between words is flexible	Whitespace and the glyphs pipe, hyphen, en dash, em dash, colon, slash and middle dot are interchangeable — see below. A name that wraps across two lines still matches.
Punctuation **inside** a word is still required	BBMRI-ERIC does not match a page writing BBMRI ERIC.
The first match in file order is reported	Put the preferred form first if a node has more than one approved name.
A variant rendering is reported	The evidence quotes what the page actually writes, so you can see that it differs from the official form without diffing by eye.
The name never changes the verdict	Point 3 stays MANUAL_REVIEW either way. The list adds an evidence line for the reviewer; it cannot produce a PASS.
A page that was never fetched is never reported as missing the name	In the 21 September run GÉANT answered HTTP 403 with no body; the evidence said the name was not looked for, not that it is absent.

All of these match a list entry written EOSC Node | BBMRI-ERIC:

```
EOSC Node | BBMRI-ERIC
EOSC Node - BBMRI-ERIC
EOSC Node – BBMRI-ERIC
EOSC Node: BBMRI-ERIC
EOSC Node BBMRI-ERIC
```

Only the body text is searched. The match runs against full_text, which excludes the HTML <title>. The EGI page's title is EGI Node - EGI, and that string appears nowhere in the body, so a name taken from the visible page. Take the name from the visible page.

Two of the matching rules above are fixes for defects, not features, and are described here so you can judge the results rather than trust them:

- **Word boundaries.** An earlier version matched a bare substring, so EGI in the list reported a match on the BBMRI-ERIC page (inside "strat**egi**c") and on Data Terra (inside "Norw**egi**an"). Neither page names the EGI node.
- **Flexible separators.** The official list writes every name as EOSC Node | X. The pages do not: BBMRI-ERIC writes a hyphen and an en dash, European DTO writes the pipe, EUDAT writes no separator at

all. Matching the pipe literally scored **0 of 9** against the real pages — a finding against every node for what is a typographic difference. Treating the glyph as interchangeable is a deliberate loosening, and it is the reason the evidence line quotes what the page actually writes. Because it is a judgement call and not a fact, it is reversible from the command line: `--strict-separators` restores literal glyph matching, and the report then states that the strict rule was in force rather than the flexible one. Against the evidence of 21 September 2026 the strict rule matches **none** of the nine nodes, which is the 0-of-9 result above. Against the published evidence of 24 September it matches none of the twelve.

A match is not proof of the right name

The default list is **unscoped**: any name in it, matching anywhere in a page's body, counts for any node. So a match tells you an approved name appears on the page — not that it is *that* page's own name. A node listing its partners would match on a partner's name.

`checklist/approved-names-scoped.txt` closes that gap. It writes each name as `node-id: EOSC Node | X`, so a name is only ever matched against the node it belongs to:

```
uv run basic-check assess --approved-names checklist/approved-names-scoped.txt
```

Running it against the published evidence of 24 September 2026 returns **the same five nodes** as the unscoped list: BBMRI-ERIC, Czechia, EUDAT, GÉANT and Slovakia. On the evidence of 21 September it returned the same two, BBMRI-ERIC and EUDAT. That is a useful result rather than a null one: the headline is not an artefact of names leaking across pages, because tying each name to its own node changes nothing.

It is **not** the default, and the reason is worth stating plainly: the *names* in it are the official ones, but the *mapping* from each name to a node id in `nodes.yaml` was derived in this repository and is not part of the Tripartite file. That mapping is the line to challenge before relying on a scoped run. The file's header comment says the same thing.

Which list was used is recorded

A report that cites a name list is only as trustworthy as your ability to tell which bytes it read. Every run records the SHA-256 of the list in `results.json` under `approved_names.sha256`, and the report prints the first twelve characters of it beside the file name. If the official list is revised and a run was made against the old one, the digests differ and you can see it without re-running anything.

```
sha256sum checklist/approved-names.txt
```

What the official list actually finds

In the published run of 28 September 2026, the committed list of thirteen names matches **four of the thirteen** nodes assessed. Every one of the four writes the name with a different separator from the list, so none of them would match literally:

Node	Outcome
BBMRI-ERIC	matched; the page writes `EOSC Node - BBMRI-ERIC`
EOSC Node Czechia	matched; the page writes `EOSC Node Czechia`
EUDAT	matched; the page writes `EOSC Node EUDAT`
EOSC Node Slovakia	matched; the page writes `EOSC Node Slovakia`
European DTO, Data Terra, PaNOSC	no match; the phrase `EOSC Node` occurs (5, 2 and 2 times), but never followed by an approved name

Node	Outcome
Poland	no match; the header logo reads "EOSC Node | Poland", and the phrase EOSC Node occurs once in the text, but never followed by an approved name
CERN, EOSC Finland, EGI, Italy	no match, and the phrase EOSC Node does not occur in the body text
GÉANT	not looked for: the page answered HTTP 403 with the Cloudflare challenge

In run web-27 (7 October 2026), eight nodes match: the same four (BBMRI-ERIC at its dev3. page, which writes EOSC Node – BBMRI-ERIC; in web-24 it was not fetched), Italy at its new address (EOSC Node | Italy, the list's own separator), Data Terra at its new address (EOSC Node Data Terra), EOSC Finland, whose page writes EOSC Node Finland (since web-25), and Poland, whose page writes EOSC Node Poland (from web-23; in web-26 it rendered only its header and footer, so there was nothing to match). GÉANT matched (EOSC Node GÉANT) in web-16 and web-22, when its page was served; in web-23 to web-33 it was not. In web-28 (9 October 2026, fifteen names) and web-31 (19 October 2026) the same eight match; in web-33 (20 October 2026, sixteen names) the new node, Germany, writes "EOSC Node Germany" and matches too: nine. Neither new node does: Life Sciences Connect's logo reads "EOSC Node | Life Sciences Connect" and the phrase EOSC Node occurs once in its text, but never followed by an approved name; EBRAINS-RI's page writes "EBRAINS RI" without EOSC Node.

CERN's new page shows "EOSC Node | CERN" only in its logo image, which is not text; its previous URL, a sign-in endpoint, had almost nothing to match against. Italy's page shows "EOSC Node | Italy" in its logo image, which is not text. GÉANT's page, when served on 24 September, wrote EOSC Node GÉANT and matched, which made five of twelve that day.

The earlier run of 21 September matched two of nine (BBMRI-ERIC and EUDAT). GÉANT was not looked for then, because it answered HTTP 403 with no body, and CERN, Czechia and Slovakia were not yet configured.

This is a finding to review, not a verdict. Point 3 remains MANUAL_REVIEW for every node either way. A non-match means the page body does not carry the approved string — which may mean the page is wrong, that the name is shown in an image or the <title> rather than in text, or that the official list is out of date. Deciding which is the reviewer's job; the tool's job is to say precisely what it did and did not find. Where nothing matched, the evidence also reports how many times the phrase the approved names share (EOSC Node) does appear, so you know where to look.

If a name you expect does not match, read full_text in that node's evidence file before assuming the page is at fault.

Adding a node

Three files change together, and the test suite refuses to let them disagree. Nothing is generated or auto-discovered: every value below is typed in by a human, on purpose, so that the report can say where it came from.

1. nodes.yaml — add the entry. Put the Node Landing Page in url, taken from the **EOSC EU Node Contributors Dashboard** field "Website address" (section 1.2, field 6), not from a search engine and not from the node's general homepage. That registered URL is what the checklist defines as the Node Landing Page, so if what you were sent differs from what is registered, the registered value wins.

Then look up eosc_page on the live index at eosc.eu/building-the-eosc-federation/ and copy the slug. Do not guess it from the node's name: the slugs are not formed consistently — Czechia is eosc-node-czechia, but the Digital Twin of the Ocean is eosc-node-digital-twin-of-the-ocean while its node id here is eosc-dto. A wrong slug makes point 4 look for a page that does not exist, and the node fails a requirement it may well satisfy.

2. checklist/approved-names.txt — add the node's approved name, exactly as the Tripartite list writes it, including the EOSC Node | prefix. This file is the official list as circulated; do not edit the wording to match what a page happens to say. If the official list has not yet been updated for the new node, leave this file alone and say so — a name absent from the official list is a real finding, and inventing an entry would hide it.

3. checklist/approved-names-scoped.txt — add the same name, prefixed with the node id you chose in step 1: cern: EOSC Node | CERN. The names must match the official file character for character; only the id prefix is added here.

Then run the suite **before** running the checker:

```
uv run pytest -q
```

The guards in tests/test_nodes.py and tests/test_checklist.py will tell you exactly which file you missed — a node with no scoped name, an id that is not a usable filename, a URL that is not https, an eosc_page that is not on eosc.eu or that points at the federation index rather than the node's own entry, two nodes sharing a landing page, or a name present in one list and not the other. These fail in milliseconds, before anything touches the network.

Only then collect the new node. Fetch just the node you added, rather than re-running the whole federation, and into a scratch directory first:

```
uv run basic-check collect --only <new-id> --results /tmp/trial
uv run basic-check assess  --only <new-id> --results /tmp/trial
```

How to move that evidence into the published run without fetching the page a second time is example 1 in section 11.

assess reads evidence per node from disk, so a node configured but not yet collected is **not** silently skipped: the run prints N node(s) have no evidence and were NOT assessed, names them, and **exits 2** while still writing the report for the rest. That exit code is the signal that the published report is incomplete — do not commit a report produced by a run that exited 2 unless you intend to publish a partial one and say so.

If you maintain your own copy elsewhere (GitLab, or a fork)

The three files above are the whole of it, and none of them is specific to GitHub. nodes.yaml and both checklist/*.txt files are ordinary version-controlled text at the repository root; edit them on whatever host you use, in a branch or straight on the default branch, and the checker behaves identically. There is no registry, no database, no service to notify, and no value cached anywhere else in the repository.

Two host-specific things do **not** carry over:

- .github/workflows/ is GitHub Actions only. On GitLab you would need a .gitlab-ci.yml expressing the same two jobs — install with uv sync --locked, run pytest, and optionally run the checker on manual trigger only. Keep the manual-trigger restraint: a scheduled compliance job means fetching other organisations' production websites on a timer.
- The committed results/ directory is this repository's published output. A separate copy will produce its own, and the two will diverge as the sites change. Decide which copy is authoritative before both are quoted in a meeting.

4. Running it

```
uv run basic-check run
```

That is the whole thing: fetch every configured node, apply the checklist, write the reports. It takes a few minutes, most of it spent deliberately waiting between requests.

The commands

```
uv run basic-check points          # the checklist, and what is decidable
uv run basic-check collect         # fetch -> results/evidence/
uv run basic-check assess          # evidence -> results/
uv run basic-check run             # collect, then assess
uv run basic-check show eudat      # one node's results in the terminal
```

Listing the configuration

Six flags on basic-check itself, used without a command, show what is configured:

```
uv run basic-check --list-nodes          # node ids, one per line
uv run basic-check --list-nodes-ids      # the same, under a second name
uv run basic-check --list-nlps           # id and Node Landing Page URL
uv run basic-check --list-approved-names # id and approved name
uv run basic-check --print-config        # all three in one table
uv run basic-check --show-node bbmri-eric  # one node's row of that table
```

--print-config prints one row per node, in nodes.yaml order, as plain aligned columns (shortened here):

Node id	Node Landing Page URL	Approved name
bbmri-eric	https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/	EOSC Node | BBMRI-ERIC
cern	https://eosc.cern/	EOSC Node | CERN
…	…	…
eosc-sk	https://eosc.sk/	EOSC Node | Slovakia

followed by two lines naming the files: *13 node(s). Ids and URLs from nodes.yaml; approved names from checklist/approved-names-scoped.txt." and "assess and run use checklist/approved-names.txt by default: the same names, unscoped, so each counts for every node."

--show-node takes one node, by id or by name in any case as --skip does (eosc-it, Italy, EOSC Node Italy), and prints that node's row of the table under the same header:

Node id	Node Landing Page URL	Approved name
bbmri-eric	https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/	EOSC Node | BBMRI-ERIC

A value that matches no node, or more than one, is an error that lists the ids. --list-nodes-ids is another name for --list-nodes, with the same output.

The approved name next to each id comes from checklist/approved-names-scoped.txt, the copy of the official list that ties each name to a node. assess and run use checklist/approved-names.txt by default, the same

names unscoped, so there each name counts for every node; the table says so under the rows. A node with no name in the scoped file shows (none). The flags can be combined (--list-nodes --list-nlps), print plain columns that are never wrapped, so they can be piped to grep or cut, and are refused together with a command. They read local files only and contact no node.

Use it before and after editing nodes.yaml or the name lists, to see what the next run will check.

Useful options

Not every option is accepted by every command, and passing one to the wrong command is an error rather than a no-op. Each table below names the commands that accept the option.

Fetching — collect and run only, because only these two touch the network:

Option	Effect
--depth N	How far to follow links. See section 5.
--delay S	Seconds between hosts (default 2.0). Raise it to be gentler.
--fetch-budget N	Depth 2 only: hard ceiling on second-hop requests for the **whole run** (default 60).
--node-timeout S	The most seconds one node may take, landing page and linked pages together (default 300; 0 = no limit). A node that hits it is abandoned: every point reads ERROR with a TimeoutError message, the browser is restarted, and the run goes on to the next node. Added on 20 October 2026, after one node's collection stalled and held the GitHub run of web-29 until its 30-minute limit.
--max-children N	Cap on followed pages per node at depth 1 (default 8). **collect only — run rejects it**, so use collect then assess if you need it.

Choosing what to check — accepted by collect, assess and run:

Option	Effect
--only a,b	Restrict which nodes are **fetched**. The report still covers all of them — see below.
--skip a,b	Leave these nodes out entirely: not fetched, not assessed, not in the report, which says so. Id or name, comma-separated or repeated — see below.
--url https://...	Check any page without editing nodes.yaml. Repeatable. With --node, the alternative page for that node.
--node id	Check one configured node at the alternative landing page given with --url, keeping its id, name and eosc_page. See below.
--eosc-page URL	With a single --url: that node's own eosc.eu page, so a point 4 failure can name the exact URL that is

Option	Effect
	missing. With --node, replaces that node's configured one.
--nodes path	Use a different node list.

Assessment and output:

Option	Where	Effect
--approved-names path	assess, run	Use this name list **instead of** the committed checklist/approved-names.txt. One name per line, ideally as node-id: Name. A path that does not exist is an error, not a fall back to the default. See section 3.
--no-approved-names	assess, run	Use no name list at all, not even the committed default. Point 3's name requirement is then not assessed.
--strict-separators	assess, run	Match the separator glyph in each approved name literally. By default a page writing a hyphen satisfies a list writing a pipe; with this flag it does not. See section 3.
--checklist path	assess, run, points	Use a different checklist version.
--run LABEL	assess, run	Label stored with the run, for telling one report from another.
--results path	all but points	Write somewhere other than results/.
--one-off	show only	Read results/one-off/ instead of the federation run.

--only narrows the fetch, not the report

Earlier versions of this tool had a real defect here, and this guide documented it as a warning: assess --only data-terra replaced the full matrix in results/ with a single row, and the resulting index.html looked like a federation report that covered one node. A short table is indistinguishable from a complete one.

That is fixed. --only now narrows **which sites are contacted**, and the report written to results/ still covers every node in nodes.yaml:

```
uv run basic-check assess --only data-terra
# One node re-assessed from fresh evidence; results.json still has every node.
```

The rows you did not select are reused from the evidence already on disk, so the report is no longer uniformly fresh. It says so itself: a **Mixed freshness** banner names the nodes that were re-fetched, states how many were not, and gives the capture date of the reused evidence. results.json records the same thing in a top-level selection list, which is [] on a full run.

An explicit --results DIR keeps the narrowing behaviour, because there nothing shared is at risk and a one-node scratch table is usually the point. Note that --results DIR relocates the whole workspace, not just the output: evidence is read from DIR/evidence as well as written there. Point it at an empty directory and you get an empty report and exit code 2, not a one-node one. Give it evidence first:

```
mkdir -p /tmp/scratch && cp -r results/evidence /tmp/scratch/evidence
uv run basic-check assess --results /tmp/scratch --only data-terra
# A deliberate one-node report, written somewhere of your choosing.
```

Or collect straight into it, which re-fetches that node rather than reusing anything:

```
uv run basic-check collect --results /tmp/scratch --only data-terra
uv run basic-check assess  --results /tmp/scratch --only data-terra
```

A bare assess still rebuilds every row from evidence on disk, so it remains the way to produce a report with a single capture date:

```
uv run basic-check assess
```

--skip leaves nodes out, and says so

--skip is for the opposite case: run on every configured node **except** some. It takes node ids or names, case-insensitively, either comma-separated or by repeating the option. Short names work too, so Italy names "EOSC Node Italy":

```
uv run basic-check run --skip Italy,Slovakia
uv run basic-check run --skip Italy --skip Slovakia        # the same
uv run basic-check assess --skip "CERN, Czechia" --skip eosc-it --skip eosc-sk
```

A skipped node is left out of the whole run. No request is sent to its site, it is not assessed, and it has no row in the report. Because it is left out on purpose, it does not count as missing evidence, so assess does not exit 2 for it. The reviewed report of 24 September was built this way, with --skip Italy, because eosc.it had no address record that day.

Unlike --only, --skip does shorten the report, including the one in results/. A shorter table must never look complete, so every report says what was left out: a **Skipped by request** banner in results.md and index.html, and a top-level skipped list in results.json ([] when nothing was skipped). Like any assess without --results, it rewrites results/, which stays under human review. For a trial run, add --results /tmp/scratch.

The option rejects values rather than guessing. If a value matches no node, the error lists the valid ids, because a typo that skipped nothing would fetch the very site you meant to leave alone. A value that names more than one node is an error too. So is skipping every node, and so is combining --skip with --url. With --only, the skip is applied after the selection: --only egi,eudat --skip egi fetches EUDAT alone.

Checking a page not in nodes.yaml

```
uv run basic-check run --url https://example.org/our-node-page
```

Ad hoc checks write to results/one-off/ so a published run is never overwritten, and the report carries a banner saying it is not a federation run. That banner matters: these reports circulate before a production decision, and a stray single-page file that looks official is a genuine hazard.

Updating one node's row: --update-results-for-node

After a node's URL changes, or when one node's page has been fixed, there is no need to re-check the whole federation. --update-results-for-node NODE, on run and assess, re-checks that node alone and replaces only its row. The stored results.json in the results directory is the memory: every other row is copied from it as it is, neither re-fetched nor re-judged.

Command	What it does
run --update-results-for-node NODE	Fetches that one page again, into a scratch directory. Only if the page loaded does it replace the node's evidence and screenshot, and its row. --accept-error merges a failed fetch too.
assess --update-results-for-node NODE	Judges the node's evidence already in evidence/ again. Offline.

Either way, results.md, index.html and results.csv are rebuilt from the stored rows, and the command prints each point's old and new verdict. NODE is an id or a name, in any case. A node the stored run skipped is inserted where nodes.yaml puts it and removed from the skipped list. The run id and time in the header stay those of the original run; results.json gains updated_at and a row_updates list with the node's previous URL, capture time and verdicts, and both reports carry an **Updated rows** banner saying which row is newer and that the others were not re-judged.

It refuses, and changes nothing, when:

- the results directory holds no results.json, or holds an alternative-URL trial (--node);
- the checklist is not the one the stored results were assessed with, or the approved-name options (--approved-names, --no-approved-names, --strict-separators) differ from theirs. One row judged by other rules would not be comparable with the rest; run the whole federation again instead;
- it is combined with --only, --skip, --url, --node, --eosc-page or --run.

The default directory is results/, the published run. Try it on a copy first:

```
cp -r results /tmp/copy
uv run basic-check run --update-results-for-node bbmri-eric --results /tmp/copy
uv run basic-check show bbmri-eric --results /tmp/copy
```

Checking a node at an alternative URL

To run every test against one configured node at a landing page other than the one in nodes.yaml, give the node with --node and the page with --url:

```
uv run basic-check run --node bbmri-eric --url https://alt.example.org/eosc-node-bbmri-eric/
uv run basic-check show bbmri-eric --one-off
```

The node keeps its id, name and eosc_page, so its scoped approved name applies to point 3 and point 4 looks for its own eosc.eu page; only the page fetched changes, and nodes.yaml is not edited. --node takes an id or a name in any case, as --skip does.

Rule	Why
Exactly one --url	It is the alternative page for that one node. Without --url, use --only to check the node at its configured page.

Rule	Why
Not with --only or --skip	The run is already one node; either would be ignored or contradict it.
Writes to results/one-off/, or another --results folder	--results results/ is refused: the evidence file is named after the node id and would replace the reviewed evidence.
--eosc-page replaces the node's eosc.eu page	For a trial of a changed registration as well as a changed page.

The reports are titled **Alternative URL check (not a federation run)** and carry an **Alternative URL, not a federation run** banner naming both addresses; results.json records the pair under alternative_url, each node record keeps configured_url, and show prints both. If the same evidence is later assessed without --node, the usual **Evidence from a different URL** warning appears, so it cannot pass as the configured page.

This is the way to try a node's new address before changing nodes.yaml (example 2 in section 11).

---

5. How deep to go: --depth

--depth	What is fetched	Requests, 9 nodes, 21 Sep 2026	Requests, 12 nodes, 24 Sep 2026	Requests, 13 nodes, 26 Sep 2026
0	The landing page only.	9	12	13
1 **(default)**	The landing page, plus links from it that could settle a checklist point — policy, contact, about, and, for item 5, policies, legal, services and resources pages. At most 8 per node, 2 per purpose.	31 (9 + 22)	44 (12 + 32)	44 (13 + 31), and the same on 28 Sep (web-10 to web-12)
2	The above, plus one further hop: a policy *index* that links on to the actual policy, for instance. At most 2 per fetched page and 1 per point, under a run-wide budget.	45 (9 + 22 + 14)	not run	not run; 63 (13 + 31 + 19) on 28 Sep, run web-13, the published run

robots.txt requests are not counted in these figures. The tool fetches one per host before touching it.

Links are followed only where the target could settle a point, so a policy link is verified rather than taken on the strength of its label.

```
uv run basic-check run --depth 2                  # bounded by the default budget
uv run basic-check run --depth 2 --fetch-budget 20  # stricter ceiling
```

A depth-2 run reports both depths. You get two summary tables — *Results at depth 1* and *Results at depth 2* — two explanatory sections, a list of every cell whose verdict differs, and a table of exactly which second-hop pages were fetched and what each was followed for.

Both tables come from the **same capture**: the shallow view is the deep evidence with the second-hop pages set aside, not a second run. Nothing is fetched twice to produce the comparison, so any difference between the two tables is the extra hop and not the page changing in between.

Is depth 2 worth it? On this federation, so far, no. Two depth-2 runs on 21 September (18 and 14 second-hop requests) each changed **zero** verdicts. That is a finding, not a disappointment: the points still marked review turn on a judgement ("clearly state") or quantify over things no crawl enumerates ("all research resources offered by the Node"), and no amount of fetching settles either kind. **Depth 2, 28 September 2026 (run web-13, checklist v3.2):** 19 second-hop pages on top of the 31 depth-1 links, and again **zero** verdicts changed: 49 PASS, 14 FAIL and 67 review at both depths under the first copy of v3.2, and 50 PASS, 8 FAIL and 72 review at both depths when re-scored against the re-issued one. **Depth 2, run web-14:** 60 second-hop pages on top of 63 direct links, and again **zero** verdicts changed (52 PASS, 10 FAIL, 68 review at both depths). That run also looked for services and policies pages at the second hop, which used the whole fetch budget before Italy and Slovakia were reached; since then those pages are looked for on the landing page only, as "pages linked by the NLP", and the second hop follows policy links alone. **Depth 2, run web-15, with that fix:** 26 second-hop pages on top of 56 direct links, 95 requests in all, and again **zero** verdicts changed (53 PASS, 9 FAIL, 68 review at both depths). **Run web-16:** 34 second-hop pages, 109 requests, and again **zero** verdicts changed (57 / 6 / 67 at both depths). **Run web-18:** 26 second-hop pages, 96 requests in the published evidence, and again **zero** verdicts changed (60 / 6 / 64 at both depths). **Run web-21:** 27 second-hop pages, 98 requests, and again **zero** verdicts changed (63 / 3 / 64 at both depths). **Run web-22:** 37 second-hop pages, 118 requests, and again **zero** verdicts changed (68 / 3 / 59 at both depths). **Run web-23:** 29 second-hop pages, 102 requests, and again **zero** verdicts changed (66 / 1 / 63 at both depths). **Run web-24:** 28 second-hop pages, 94 requests, and again **zero** verdicts changed (60 / 1 / 59 / 10 at both depths). **Run web-25:** 34 second-hop pages, 109 requests, and again **zero** verdicts changed (68 / 0 / 62 at both depths). **Run web-26:** 34 second-hop pages, 109 requests, and again **zero** verdicts changed (67 / 0 / 63 at both depths). **Run web-27:** 33 second-hop pages, 109 requests, and again **zero** verdicts changed (69 / 0 / 61 at both depths). The second hop reached governance, secretariat and news pages, which is also where it met a personal name that depth 1 had never collected (section on masking). Run it yourself before assuming the same holds for another federation or a later date.

Be considerate with other people's servers

These are production sites that did not ask to be tested. The defaults are deliberately restrained: one request per node every two seconds, a per-node cap on followed links, and a single run-wide budget for second-hop requests rather than a per-node one.

This is not theoretical. geant.org served HTTP 200 at 12:43 UTC on 21 September, HTTP 403 at 13:07 the same afternoon after several runs, redirecting to a Cloudflare bot-protection challenge, and HTTP 200 again at 18:39 from a GitHub runner — a different address that had made no requests that day. It happened again on the 25 September trial, at 04:42 UTC: GÉANT answered HTTP 403 with the same challenge, and five of its cells that were decided on 24 September moved to review. **Repeated automated runs are visible to the sites you are checking**, and the block tracks the requesting address rather than the tool's identity. Run the full

suite when you need a result, not in a loop, and use assess, which needs neither browser nor network, when you are iterating on the report itself.

---

6. Reading the output

results/ after a run:

File	What it is
index.html	**The deliverable.** The matrix, per-node detail, and the full checklist text. Open it in a browser.
results.md	The same content as Markdown, for reading on GitHub or pasting into a document.
results.csv	One row per node-point, for a spreadsheet or pivot table.
results.json	The full structured run, including every piece of evidence. Use this for any further analysis.
checklist-v3.2.html	The checklist points in full, with the reasoning behind each verdict.
evidence/<id>.json	The raw capture for one node: links, images, controls, text, headers.
evidence/screenshots/<id>.png	What the page looked like when it was fetched — the **visible viewport only**, so footers and anything else below the fold are not in the image. Do not read a missing element here as absent from the page; check links and full_text in the evidence JSON.

The four verdicts

	Meaning
🟢 **PASS**	Satisfied, and the tool can show why.
🔴 **FAIL**	Violated, and the tool can show why.
🟠 **review**	A human must decide. The tool has gathered the evidence and refuses to guess.
🟣 **ERROR**	Could not be assessed at all.

ERROR is not a finding against the node. It means the tool has no page to judge: the host did not resolve, the request failed, or robots.txt asked crawlers to stay away and the tool complied. The reason is in each cell's message, for example The page could not be fetched: not fetched: robots.txt disallows it, and robots_note in the evidence file records what robots.txt said. In the 24 September run only Italy would have been ERROR, and it was skipped instead; the published run of 26 September has no ERROR. On the 25 September trial, BBMRI-ERIC's new dev3. address was ERROR on all ten points because its robots.txt disallows every path.

A FAIL is an accusation, so the tool is careful about making one. Absence is never concluded from a page that did not render: if a capture yields no links at all, or fewer than ten DOM elements, or under 500

characters of text, the tool returns review rather than FAIL. The point is that "we found no contact link" and "the page did not load for us" must not produce the same verdict.

The same restraint applies to being blocked. GÉANT's HTTP 403 moved four cells from decided verdicts to review — the tool does not convert bot protection into a compliance failure. A later run from an unthrottled address reached the page and settled all four, which is the restraint paying off: had the block been recorded as failure, that report would have contained four wrong verdicts instead of four honest abstentions. The block came back in the published run of 26 September, and the same four cells are review there.

Reproducing a verdict by hand

Every verdict is traceable to evidence on disk:

```
uv run basic-check show geant                  # verdicts and evidence in the terminal
python3 -m json.tool results/evidence/geant.json | less   # the capture, personal data masked
```

results.json carries the same evidence strings the report displays, so you can check any claim without re-fetching anything.

To work out *why* a particular cell came out the way it did, the decision procedure for every point — every branch, in evaluation order, with the verdict it yields — is tabulated in section 5 of the **analysis workflow**. Section 6 of that document lists the cases where the tool is known to be wrong, which is the first place to look when a verdict surprises you.

Personal data is masked

Contact pages sometimes name individuals, such as a media officer with a direct line. The checklist needs none of that: point 6 asks for a way to reach the node helpdesk, which is a role. So src/basic_check/privacy.py masks personal data at the two places anything is written to disk. When evidence is collected, the evidence file is masked before it is saved. When the reports are written, the run is masked again before any of the four formats is rendered, so a report rebuilt from evidence captured before masking existed is masked too.

The table shows the shapes that are recognised, not real entries. firstname, lastname and CC (a country code) stand for whatever is on the page, and N for any digit.

Found on the page	Written as
firstname.lastname@example.org, firstname.lastname [at] example.org	XXXXX@example.org, XXXXX [at] example.org
support@egi.eu, it@helpdesk.bbmri-eric.eu, geant@geant.org	unchanged: role and organisation mailboxes
+CC NN NNN NNNN, tel:00CCNNNNNNNNN (dialled with 00)	+CC XXXXXX, tel:+CC XXXXXX
tel:+CC(0)NN%20NNNNNN (a space written %20 inside a link)	tel:+CC XXXXXX
Tel.: NN NNNN NNNN, tel. (NN) NNN NNNN (no prefix, but labelled)	Tel.: XXXXXX, tel. XXXXXX
Dr. Firstname Lastname, lastname@… (a name next to the address)	Dr. XXXXX XXXXX, XXXXX@…

An address is kept only when it is recognisably a role, such as support, info, helpdesk, csirt or any mailbox under a helpdesk. host. Every other address is masked. A name is masked only near a masked address, and

never inside a URL or as an all-capitals acronym. Office switchboards are masked like any other number, because nothing in a number says who answers it. Masking changes no verdict, because every check works from the parts that are kept. Screenshots are not masked; they show the visible part of the landing page only, not the contact pages where these details appeared.

The published run is masked. Run web-33 (20 October 2026) was collected with masking in place, and a search of all its evidence and reports found no personal address or phone number. Two personal names that the masking does not cover were removed by hand from the evidence, each replaced by a placeholder of the same length so every character count and verdict stays as collected: a quotation attribution on CERN's page (in cern.json and in its screenshot) and the node coordinator named on EGI's page (in egi.json, in the link text and, since web-12, also in the link's address, /person/.../, which earlier published runs still carried). The second hop of web-13 added a third: a person quoted in a news item on a Czechia page reached only at depth 2 (in eosc-cz.json, in the quotation and in the address of the interview it links to). web-14, which also reads services and policies pages, added many more, and web-15, web-16, web-18, web-21, web-22, web-23, web-24, web-25, web-26, web-27, web-28, web-31 and web-33 carry the same ones (BBMRI-ERIC's apart, in web-24, where its page was not fetched; web-25 adds fifteen on BBMRI-ERIC's dev3. pages: its Director General on the legal notice, the seven members of its scientific and ethical advisory board, the Stakeholder Forum chair and the head of public affairs, and five finance, project and HR staff listed in capitals) (web-21 adds one, a photo credit in an image description on Data Terra's new page; web-22 adds two staff names in a news item on EUDAT's new page and one more GÉANT observer; web-27 adds the people named in the captions of historical public-domain photos on Poland's page; web-28 adds, on EBRAINS-RI's pages, its CEO, a section head, its press contact, three testimonial authors and two people named in news items and image captions, also in image file names and link addresses, and the coordinator named in a video title in Life Sciences Connect's screenshot; web-31 adds the author credited in the document log of BBMRI-ERIC's Acceptable Use Policy and a person interviewed in an EBRAINS-RI news item): about thirty staff, board and news-author names on BBMRI-ERIC's and EGI's pages, and the author lists of publication citations on Czechia's, D4Science's (EOSC DTO, Italy) and PaNOSC's pages. Two of those citation lists were already in the evidence published as web-13 and were missed then; they remain in the git history. web-16, the first run with GÉANT's page served since 24 September, reached its executive team, governance and membership pages: about 120 names of officers, board members and General Assembly representatives, some cut short at the end of a link label ("Firstname Lastna…"). They were taken from the fixed patterns of those pages ("Representative:", "Substitute:", "… Chief … Officer", "… Member") and masked everywhere, truncated forms included. A deeper run reads more pages, and so needs this check more; look for citations ("Surname, I." lists before a year) as well as for people named in prose. The masking handles addresses and phone numbers only, so a new run needs the same check for names. Before it, commit ada1b4a (25 September 2026) masked the 24 September run's results/evidence/ and rebuilt the reports offline. All 120 verdicts and the 44 / 6 / 70 tally were unchanged. A few character counts in the report dropped slightly, by the length of the masked text (BBMRI-ERIC's page from 7100 to 7094 characters). The first masking pass had let two phone formats through, the %20 and the bracketed area code in the table above. Both were found by searching every published file for phone-shaped text, fixed, and added to the tests. tests/test_privacy.py now also checks the committed evidence itself, and fails if any file in results/evidence/ contains a personal address or a phone number. The earlier, unmasked versions remain in the git history. Removing them would mean rewriting the history of main, which has not been done.

To mask evidence collected before masking existed, for instance a trial run in a scratch directory, re-write it with the same function and rebuild:

```
uv run python -c "
import json, pathlib
from basic_check.privacy import mask_data
for p in pathlib.Path('/tmp/trial/evidence').glob('*.json'):
```

```
    p.write_text(json.dumps(mask_data(json.loads(p.read_text())), indent=2, ensure_ascii=False))
"
uv run basic-check assess --results /tmp/trial --run <run id>
```

Rebuild with the nodes.yaml the run was collected with (--nodes), or a URL changed since then is shown against evidence taken from the old one (section 3, "Changing a node's URL").

---

7. The test suite

```
uv run pytest -q              # 462 tests, offline, a few seconds
uv run pytest -v              # names of every test
uv run pytest tests/test_checks.py  # one file
uv run pytest -k depth            # anything about depth
uv run ruff check src tests       # lint
```

File	Tests	Covers
test_checklist.py	19	The transcription matches the source document, including its SHA-256 for every committed revision, the scoped name list agrees with the official one, switching DEFAULT_CHECKLIST carries the help texts with it, and each revision selects its own item 5 rule.
test_checks.py	83	The verdict logic, point by point, including both item 5 rules (v3.2: no AUP/UAP link on the NLP fails, one document for both and a licence are review), including the render gate, the EOSC-asset token rule, that a point 3 summary never denies having a name list it was given, and that a link merely labelled "support" does not settle point 6.
test_cli.py	113	Command wiring, options, ad hoc --url isolation, that --only does not shrink the published report, that --skip leaves nodes out and says so, that --help stays complete, and that evidence from another URL than the configured one is flagged.
test_crawl.py	35	Link selection, including policy words found only in a hyphenated address, host containment, depth-2 budget, the depth-1 view.

File	Tests	Covers
test_names.py	64	Parsing, scoping, word boundaries, separator flexibility and its strict counterpart, and the recorded digest.
test_nodes.py	14	nodes.yaml itself, and that the default copy shipped for --restore-default-config is the committed file: every node declares every field, ids are unique and usable as filenames, URLs are absolute https, no two nodes share an eosc_page, no eosc_page is the federation index, and every node has a scoped approved name.
test_privacy.py	51	Personal-data masking: personal addresses and phone numbers masked, role mailboxes kept, the name beside an address masked but not a title or an acronym, that both the evidence files and every report format are written masked, and that the committed evidence stays masked.
test_report.py	37	Matrix rendering, the column legend that follows the item 5 rule, the dual-depth tables, the mixed-freshness banner, the name-list provenance, and input that would break a table or a list — a `
test_update_row.py	22	--update-results-for-node, on a copy of the committed results/: only the named row changes, the reports gain one **Updated rows** line, run collects exactly one node, a failed fetch changes nothing unless --accept-error, a skipped node is inserted in nodes.yaml order, and every refusal.

The suite makes no network requests and needs no browser, which is why CI runs it without downloading Chromium.

A fuller treatment — why the suite takes a few seconds rather than a fraction of one, the testing criteria behind it, and the defects that each earned a regression test — is in the **test suite and configuration overview**.

Test names are sentences, because a failing test should tell you what broke without opening the file:

```
test_a_consent_banner_does_not_excuse_a_missing_link
test_the_two_tables_show_their_own_verdicts
test_a_pipe_in_a_url_cannot_break_the_table
```

A note on what a passing suite means

Green means nothing objected. It does not mean everything was verified.

When adding a check, make the test fail first, then implement — and when changing a guard, mutate it deliberately to confirm the test actually bites. A test that cannot fail is worse than no test, because it produces confidence without evidence. Both dual-depth guards in test_report.py were confirmed this way: rendering the deep results in the shallow table, and forcing the stray heading back, each fail exactly one test.

---

8. Continuous integration

.github/workflows/tests.yml runs on every push and pull request, and can also be started by hand: uv sync --locked, ruff check, then pytest with a JUnit report published to the run summary and uploaded as an artifact. No browser is installed, because the suite does not need one.

Both workflows use --locked rather than a plain uv sync. The difference matters: plain uv sync quietly updates the lockfile when it disagrees with pyproject.toml and carries on, so a dependency change could go green in CI while the versions tested were not the versions committed. --locked installs exactly what uv.lock pins and fails otherwise. If CI ever stops at the install step saying the lockfile needs updating, that is the guard working — run uv lock and commit the result.

.github/workflows/compliance.yml is **manual dispatch only** — deliberately. A scheduled compliance run would mean fetching every node's production website on a timer, which is exactly the behaviour that got GÉANT's bot protection to start refusing requests. The workflow reads nodes.yaml from the branch it runs on, so on main today it fetches BBMRI-ERIC's dev3. address, the one the published run was collected from.

It takes three inputs: only (comma-separated node ids, empty for every node), depth (1, 0 or 2) and delay (seconds between nodes). depth: 2 was added after a web run could not reproduce the published depth-2 report — the input had offered only 1 and 0, so the workflow could not produce the report the repository publishes. Choose it when you want that like-for-like comparison, and note it means roughly fourteen further requests to other people's sites.

The workflow has permissions: contents: read and does **not** commit its output. The reports are uploaded as the compliance-results artifact, and republishing is a deliberate local step: download the artifact, review it, and commit. A workflow that could rewrite the published verdicts unattended would remove the review that makes them worth publishing.

Its run summary publishes **counts only** — how many nodes, and the tally of verdicts — labelled as unreviewed automated output, with the per-node detail left to the compliance-results artifact. On a public repository the run summary is world-readable, and while the committed report is public anyway, a table of FAILs against named organisations generated automatically under the repository's name reads as a finding rather than as a draft. The summary also declines to report anything unless the check step succeeded, because results.json is committed: a failed run would otherwise have read the previous reviewed run's counts out of a fresh checkout and published them under a new run number.

---

9. Troubleshooting

Symptom	Cause and fix
Executable doesn't exist ... playwright install	Chromium is not installed. Run uv run playwright install chromium --with-deps. Not needed for assess, show, points or pytest.
The report has fewer nodes than you expected	You passed --results DIR together with --only, which narrows both deliberately. Without an explicit --results, --only narrows the fetch alone and the report keeps every node. Section 4.
The report carries a **Skipped by request** banner, or has fewer rows than nodes.yaml	Expected after --skip: those nodes were left out on purpose, and the banner and results.json name them. Run without --skip for every node. Section 4.
The report carries a **Mixed freshness** banner	Expected after --only: the nodes you did not select were reused from evidence on disk. Run a bare uv run basic-check collect then assess for a report with one capture date. Section 4.
Every point for a node is ERROR, with the message not fetched: robots.txt disallows it	The site's robots.txt excludes the tool, and it complied. Common on staging hosts: BBMRI-ERIC's dev3. address did it until 5 October 2026. Ask the node for the public address or an exception; do not work around it. Section 6.
assess warns that nodes were assessed from evidence collected at a different URL, and the report carries an **Evidence from a different URL** banner	A node's url in nodes.yaml changed after its evidence was collected, so the row shows the new address above the old page's verdicts. Collect that node again, or rebuild with the old node list via --nodes. Section 3, "Changing a node's URL", and example 2 in section 11.
test_the_committed_evidence_publishes_no_personal_address_or_phone fails	Evidence that is not masked has reached results/evidence/, usually copied in from a run made before masking existed. Mask it with mask_data and rebuild (section 6). Do not commit until the test passes.
A node shows HTTP 403 and everything moved to review	Bot protection, not an access policy. Check final_url in the evidence for a __cf_chl_rt_tk parameter. Wait, run less often, or verify that node by hand in a browser.
Everything is review for one node	The capture probably did not render. Look at evidence/screenshots/<id>.png — that is exactly what the render gate is protecting you from.
test_checklist.py fails on a hash	The checklist PDF changed. That is the test doing its job: transcribe the new version into a new YAML file rather than adjusting the hash.
A run takes far longer than expected	--delay defaults to 2.0s between hosts and slow nodes are waited on. This is intentional.
Point 3 reports NONE of the ... approved name(s) ...	Expected for many nodes: in the published run the

Symptom	Cause and fix
appear for a node you know is named correctly	committed list matches 4 of 13. The <title> is not searched, and a name is not matched inside a longer word — though the separator between words is flexible. Read full_text in that node's evidence file. Section 3.
Point 3 says no approved name was supplied for this node	Your list is scoped and has no node-id: line for that node. Add one, or use a bare name to cover every node. Section 3.
Point 4 says PASS but you cannot find the link	Look in the footer. The check reads the DOM, not the visible area, and several nodes put the eosc.eu link in a legal/navigation column at the very bottom. The exact URL is quoted in the evidence — search the page for it rather than scanning by eye. Note also that the target is never requested, so the check cannot tell you the page still exists.
Point 3 says No approved-name list was used	You passed --no-approved-names, or checklist/approved-names.txt is missing from your checkout (the run warns on stderr when it is). Section 3.
The report says the name list came from a list supplied for this run when you expected the official one	You passed --approved-names. Drop the flag to use the committed default. Section 3.
Point 3 says a name was not looked for	The page returned no body — a 403 or a failed render. Fix the collection first; the name says nothing until there is a page to read. Retrying later, or from a different address, is often enough.
A match is reported from the unscoped list and you want a firmer claim	Prefix each name with its node id so it is only matched against that node. Section 3.

10. Extending it

- **Add a node:** three files together — nodes.yaml, checklist/approved-names.txt and checklist/approved-names-scoped.txt — then pytest, then collect --only <id>. Full procedure and failure modes in Adding a node in section 3; the commands through to publication in example 1, section 11.
- **Change a node's URL:** the url line in nodes.yaml only. Example 2, section 11.
- **Add a checklist version:** new YAML beside v3.2.yaml, with source_file and source_sha256; do not edit an existing version in place. Then change DEFAULT_CHECKLIST in src/basic_check/cli.py. Example 3, section 11.
- **Add a check:** implement in src/basic_check/checks.py, write the failing test first, and prefer returning MANUAL_REVIEW with good evidence over a confident guess. Document its branches in ANALYSIS-WORKFLOW.md — a check whose decision procedure is not written down cannot be reviewed.
- **Change the report:** src/basic_check/report.py renders HTML, Markdown and CSV from one run dict. tests/test_report.py covers the matrix; add to it, because a rendering bug is silent.

Text that reaches the Markdown report is escaped at the boundary. Two helpers in report.py do it, and new rendering code should use them rather than interpolating a value directly:

Helper	Use for	What it does
_md_cell(value)	Anything placed in a table cell	Escapes the pipe character so it cannot open a column, and flattens newlines
_md_text(value)	Inline text — list items, headings, bold runs	Flattens newlines so the element does not end early

Both collapse runs of whitespace, which leaves ordinary text untouched. A pipe in a **URL** is the exception: inside an autolink a backslash is not an escape, so pipes there are percent-encoded to %7C instead.

The reason this is handled centrally is that a rendering bug of this kind is silent — the report is still valid Markdown, just with the columns shifted or a sentence promoted out of its list item, and nothing raises. Node names come from nodes.yaml, evidence strings from fetched pages, and point titles from a checklist transcription, so none of it is under this module's control.

11. Worked examples

Three step-by-step procedures, one for each change that comes up most often: adding a node, changing a node's URL, and adopting a new revision of the checklist. Each lists which files to edit, the exact commands, and how to put the result into the published results/ directory.

Four rules apply to all three:

- **Try it in a scratch folder first.** --results /tmp/trial writes nowhere near results/, so nothing published changes until you decide it should.
- **Run uv run pytest -q after every edit.** It needs no network, takes a few seconds, and names the file you got wrong.
- **Contact as few sites as possible.** Only collect and run send requests. assess works on saved evidence, so it can be repeated as often as you like.
- **A new published run needs a human review before it is committed.** The tool's verdicts are not a compliance statement (section 6). Write the review down, as results/REVIEW-2026-09-24.md does for the 24 September run, and commit only then.

The run labels below (live-2026-10-01-...) are examples. Use the date of your own run.

Example 1 — adding a node and publishing it

The example adds a node with the placeholder id eosc-example. Replace every value with the real ones.

Step 1: edit three files. All three are at the repository root or in checklist/, and all are edited by hand.

nodes.yaml: add the entry. url is the landing page as registered in the EOSC EU Node Contributors Dashboard ("Website address", section 1.2, field 6). Copy eosc_page from the federation index; do not guess it.

```
- id: eosc-example
  name: EOSC Node Example
  url: https://eosc-node.example.org/
  eosc_page: https://eosc.eu/building-the-eosc-federation/eosc-node-example/
```

checklist/approved-names.txt: add the approved name exactly as the Tripartite list writes it:

```
EOSC Node | Example
```

checklist/approved-names-scoped.txt: add the same name, with the node id in front:

```
eosc-example: EOSC Node | Example
```

If the Tripartite list does not include the node yet, do not make a name up. Leave both name files alone. test_nodes.py::test_every_node_has_a_scoped_approved_name then fails and names the node, which is intended: the node is not ready to be published. In the meantime, check it in a scratch folder with --url https://eosc-node.example.org/ (see Checking a page not in nodes.yaml, section 4), which needs no configuration.

Step 2: check the configuration, offline.

```
uv run pytest -q
```

A missing scoped name, an id that is not a valid filename, a URL that is not https, an eosc_page outside eosc.eu, or a landing page already used by another node each fail with a message naming the problem.

Step 3: trial run for the new node only. This contacts one site.

```
uv run basic-check collect --only eosc-example --results /tmp/trial
uv run basic-check assess  --only eosc-example --results /tmp/trial
uv run basic-check show eosc-example --results /tmp/trial
```

Open /tmp/trial/index.html or /tmp/trial/results.md and read the new row. If the page returned ERROR (robots.txt, a 403, a DNS failure), stop there. An ERROR row is not a result worth publishing.

Step 4: put the node into the published run. There are two ways.

A — add the one node to the current run (one site contacted, and already done). Reuse the evidence from step 3 rather than fetching the page again:

```
cp /tmp/trial/evidence/eosc-example.json results/evidence/
cp /tmp/trial/evidence/screenshots/eosc-example.png results/evidence/screenshots/
uv run basic-check assess --only eosc-example --skip Italy \
  --run live-2026-10-01-plus-eosc-example
```

The report covers every node. A **Mixed freshness** banner says that only eosc-example is new and that the other rows are reused from the earlier capture, with its date. The published run includes Italy, read at https://eosc.it/ since 1 October 2026, so no --skip is needed; skip a node only while a trial collection (collect --only <id> --results /tmp/trial) shows its page does not load.

B — a complete new run (every node contacted once). Use this when the existing evidence is old enough that a single timestamp is worth more than leaving the other sites alone:

```
uv run basic-check run --skip Italy --results /tmp/new-run --run live-2026-10-01
# review /tmp/new-run/results.md and index.html, then:
rm -rf results/evidence
cp -r /tmp/new-run/evidence results/evidence
uv run basic-check assess --skip Italy --run live-2026-10-01
```

The last command re-renders the reports in results/ from the evidence you reviewed. It makes no requests.

Either way, assess must end with **exit code 0**. Exit 2 means a configured node has no evidence, so the table is incomplete.

If assess warns "Evidence from a different URL". It means a node's URL in nodes.yaml is not the page its evidence was collected from. That is the case for BBMRI-ERIC from 25 to 26 September 2026, when nodes.yaml had the dev3. address but the published evidence was from the www. one. The warning also appears as a banner in both reports. Either collect that node again first (example 2), or assess with the node list the evidence was collected with, plus the new entry:

```
git show 53081f6:nodes.yaml > /tmp/nodes-published.yaml
# append the eosc-example entry from step 1 to /tmp/nodes-published.yaml
uv run basic-check assess --nodes /tmp/nodes-published.yaml --only eosc-example \
  --skip Italy --run live-2026-10-01-plus-eosc-example
```

Step 5: re-run the tests against the new published run.

```
uv run pytest -q
```

Two tests read the committed evidence. Each exists to make a change visible, not to be satisfied mechanically:

- test_privacy.py::test_the_committed_evidence_publishes_no_personal_address_or_phone must pass as it stands. If it fails, personal data is about to be published. Stop and find out why; never change the test to let it through.
- test_names.py::test_the_official_names_match_the_nodes_that_show_them pins which nodes display their approved name (four for the 26 September run; five on 24 September, when GÉANT's page was served). If the new page shows its name, add its id to the expected set and update the docstring to say why.
- test_update_row.py::test_published_results_are_the_run_these_tests_copy pins how many nodes the run has and which were skipped (13 and none, for run web-13). Update it to the new run's shape.

Step 6: update what quotes the published figures, then commit. results/ is regenerated, but these are written by hand:

- a review document for the new run, like results/REVIEW-2026-09-24.md;
- the headline tally in README.md ("130 cells: 69 PASS · 0 FAIL · 61 review");
- "The published figures" in docs/TEST-SUITE.md, and the node list in "The node list — a YAML file" in the same file.

```
git status
git add nodes.yaml checklist/approved-names.txt checklist/approved-names-scoped.txt \
      results README.md docs tests
git commit -m "Add eosc-example; publish run live-2026-10-01-plus-eosc-example"
git push
```

Example 2 — changing a node's landing page URL

The example is the change actually made on 25 September 2026: BBMRI-ERIC moved from https://www.bbmri-eric.eu/eosc-node-bbmri-eric/ to https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/. It was set back on 26 September, because the dev3. host's robots.txt excludes the checker, so nodes.yaml has the www. address again.

Which files change. One line in one file: url for that node in nodes.yaml. Nothing else in the repository stores a node's URL. Add a comment recording the old address and the date, so the published evidence can still be traced:

```yaml
- id: bbmri-eric
  name: BBMRI-ERIC
  # URL changed on 25 September 2026, from
  # https://www.bbmri-eric.eu/eosc-node-bbmri-eric/
  url: https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/
  eosc_page: https://eosc.eu/building-the-eosc-federation/eosc-node-bbmri-eric/
```

Change eosc_page as well only if the node's own entry on eosc.eu moved; a new landing page does not usually mean a new eosc.eu entry. The name files change only if the node's approved name changed. Keep the id: it names the evidence file, and changing it would make the node look new.

Step 1: check the configuration.

```
uv run pytest -q
```

Step 2: trial run for that node only. This contacts one site.

```
uv run basic-check collect --only bbmri-eric --results /tmp/trial
uv run basic-check assess  --only bbmri-eric --results /tmp/trial
uv run basic-check show bbmri-eric --results /tmp/trial
```

Check that the page actually loaded before going further. On 25 September the dev3. host's robots.txt disallowed every path, the tool honoured it, and all ten points were ERROR. A result like that is not worth publishing: ask the node for its public address, or for the checker to be allowed, and stop there.

Step 3: update that node's row in the published run. Only BBMRI-ERIC is fetched again; the other rows are copied from results/results.json as they are, neither re-fetched nor re-judged:

```
uv run basic-check run --update-results-for-node bbmri-eric
```

That fetches the one page into a scratch directory, and only if the page loaded does it replace bbmri-eric's evidence, screenshot and row, then rebuild results.md, index.html and results.csv from the stored rows. A failed fetch changes nothing (add --accept-error to record it anyway). It prints each point's old and new verdict, and both reports gain an **Updated rows** banner naming the node, the date and the previous URL. The run id and time in the header stay those of the original run. To reuse the evidence from step 2 instead of fetching again, copy it into results/evidence/ and run uv run basic-check assess --update-results-for-node bbmri-eric, which is offline. To see the outcome before touching the published run, try it on a copy first: cp -r results /tmp/copy and add --results /tmp/copy. For a completely fresh table instead, follow path B of example 1 step 4.

Why the order matters. assess labels each row with the URL in nodes.yaml, but takes the verdicts from the evidence on disk. Between step 1 and step 3, the two describe different pages. assess now detects this. It prints a warning naming the node and both URLs, puts an **Evidence from a different URL** banner in results.md and index.html, and records the pair under url_mismatch in results.json. Step 3 clears it, because the evidence now comes from the configured URL.

To rebuild a run collected with an older node list, without the warning, use that list. For the reviewed run of 24 September (in the git history at 800d632), assessed in a scratch copy of its results/:

```
git show 53081f6:nodes.yaml > /tmp/nodes-2026-09-24.yaml
uv run basic-check assess --run live-2026-09-24-no-italy --skip Italy \
  --nodes /tmp/nodes-2026-09-24.yaml
```

Step 4: tests, review, commit. As in example 1, steps 5 and 6: run uv run pytest -q, check the two tests that read the committed evidence, write the review, update the quoted figures, and commit nodes.yaml, results/ and the documents together.

Example 3 — moving to a new revision of the checklist

There is no URL to change. The tool never downloads the checklist document. A copy of it is committed in checklist/, and three things connect the tool to it:

Where	What it says
src/basic_check/cli.py, the line DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.2.yaml"	Which checklist revision is used when --checklist is not given. **This is the one line to change** to make a new revision the default. The help texts, the tests and the reports all follow it.
checklist/vX.Y.yaml: checklist_version, checklist_date, source_document, source_file, source_sha256	Which document that revision was transcribed from, and its SHA-256 hash.
--checklist / -c <file> on assess, run and points	A different revision for a single run, without changing the default.

When the originating document changes, you therefore add a new file for the new revision and point the default at it. v3.2.yaml is never edited: the published reports name the file they were produced from.

The example assumes a v3.3 dated 15 October 2026. Use the real version, date and filenames. This procedure was last applied on 29 September 2026, to move from v3.1 to v3.2 (and before that on 26 September, from v3.0 to v3.1); the header of checklist/v3.2.yaml records what changed, point by point.

Step 1: commit the new document next to the old one.

```
cp ~/Downloads/20261015_Node_Landing_Page_Verification_Checklist_v3.3.pdf checklist/
sha256sum checklist/20261015_Node_Landing_Page_Verification_Checklist_v3.3.pdf
# on macOS: shasum -a 256 checklist/20261015_Node_Landing_Page_Verification_Checklist_v3.3.pdf
```

The committed v3.2 file is a PDF rendering of the circulated .docx. Commit the .docx itself if you prefer: source_file may name either.

Step 2: create the new revision's file from the old one.

```
cp checklist/v3.2.yaml checklist/v3.3.yaml
```

In checklist/v3.3.yaml, change the header:

```
checklist_version: "3.3"
checklist_date: "2026-10-15"
source_document: 20261015_Node_Landing_Page_Verification_Checklist_v3.3.docx
source_file: 20261015_Node_Landing_Page_Verification_Checklist_v3.3.pdf
source_sha256: <the hash printed in step 1>
```

The version must match the filename (v3.3.yaml declares "3.3"); a test enforces this.

Step 3: compare every point with the new document. No tool can do this step for you. For each point in v3.3.yaml:

- If the wording changed, update title and requirement, which are quoted word for word. Then read the check_* function named in implemented_by, in src/basic_check/checks.py, and change it if it no longer applies the new wording.
- If what a script can decide changed, update decidable and decidable_note.
- If a point was **added**, write a new check_<id> function in checks.py, call it from run_all() in checklist order, name it in the point's implemented_by, and add tests for it to tests/test_checks.py.
- If a point was **removed**, delete its entry, its function and its tests.
- If a rule became **stricter or looser**, older runs must keep the rule they were made under. Do not simply change the function: give it a switch, pass the switch from run_all(), and declare it in the new YAML only, so the older files select the old rule by its absence. v3.2 did this for item 5: item5_on_nlp: true in v3.2.yaml, read by _item5_on_nlp in cli.py, selects the rule that the AUP and UAP must be linked from the NLP, and v3.0.yaml/v3.1.yaml, which lack the key, keep "on the NLP or in the Catalogue". Add tests for both rules, and rebuild the published run with the old file to prove it is unchanged.

Step 4: try the new revision offline, before switching. -c applies it to one run only. Re-scoring saved evidence contacts no one:

```
uv run basic-check points -c checklist/v3.3.yaml
mkdir -p /tmp/v33 && cp -r results/evidence /tmp/v33/
uv run basic-check assess -c checklist/v3.3.yaml --results /tmp/v33 --run trial-v3.3
```

Compare /tmp/v33/results.md with results/results.md. Any difference comes from the new rules alone, since the evidence is the same. If the published run skipped a node or used an older nodes.yaml, pass the same --skip and --nodes here.

Step 5: make it the default, and run the tests. In src/basic_check/cli.py:

```
DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.3.yaml"
uv run pytest -q
uv run basic-check --help     # now says "against checklist v3.3"
```

The tests check the new file's hash, that its version matches its filename, and that every point maps to exactly one check_* function and every function to one point. They also check that v3.2.yaml and its document are still intact, so older runs can still be rebuilt with -c checklist/v3.2.yaml. What they cannot check is whether a rule is right: that is step 3.

Step 6: produce and publish the results. The default revision is now v3.3, so no -c is needed. Choose between the same two paths as in example 1:

```
# fresh evidence from every node:
uv run basic-check run --results /tmp/new-run --run live-2026-10-20-v3.3
# review, then:
rm -rf results/evidence && cp -r /tmp/new-run/evidence results/evidence
uv run basic-check assess --run live-2026-10-20-v3.3
```

Alternatively, re-score the published evidence against v3.3 without contacting anyone, as in step 4 but writing to results/. The report then says v3.3, while the capture date is still that of the evidence.

The reports now link to results/checklist-v3.3.html. The old results/checklist-v3.2.html stays behind; remove it, and update the places that name v3.2 in prose or links:

```
git rm results/checklist-v3.2.html
rg -n "v3\.2" README.md docs checklist/README.md .github
```

In README.md, the table of checklist points links each point to results/checklist-v3.2.html#p…. Those links break once the file is removed, so change them to v3.3.

Step 7: tests, review, commit. As in example 1, steps 5 and 6. If any check_* function changed in step 3, also update its decision procedure in section 5 of ANALYSIS-WORKFLOW.md, which describes each point's rule branch by branch. Commit the new document, checklist/v3.3.yaml, cli.py, any changed checks and tests, results/ and the documents together, so the commit shows the whole change.

Sources

- Checklist: checklist/20260928_Node_Landing_Page_Verification_Checklist_v3.2.pdf, transcribed to checklist/v3.2.yaml with its SHA-256 pinned (earlier revisions: checklist/v3.1.yaml and checklist/v3.0.yaml, with their documents)
- EOSC Federation node index: https://eosc.eu/building-the-eosc-federation/
- Repository: https://github.com/marioreale/eosc-basic-compliance
- uv documentation: https://docs.astral.sh/uv/
- Playwright for Python: https://playwright.dev/python/